

Автоматизированное тестирование. Selenium

Особенности BDD

- Фокусировка на функциональности
- Помогает вести разработку начиная со стадии оформления требований
- Тестирование методами черного ящика для юзер-сценариев
- Имитация действий пользователя

TDD vs. BDD

TDD

- Нет четкого критерия для остановки написания тестов
- Сложно применять для функциональных UI-тестов

BDD

- ▶ Тесты пишутся для каждой пользовательской истории (user story) .feature
- ▶ Хорошо применимо для написания функциональных тестов
- ▶ Написание интеграционных сценариев в формате end-to-end

Selenium

Установка и настройка

Для установки и настройки Selenium WebDriver на локальной машине понадобятся:

1. Java (<http://www.java.com/en/download>).
2. Selenium server (скачать standalone версию можно тут — <https://www.selenium.dev/downloads/>). Latest stable version **3.141.59**
3. ChromeDriver (для тестирования в Google Chrome). Качается отсюда: <https://chromedriver.chromium.org/downloads>

Установка и настройка

```
ghghost — java -jar ~/Downloads/selenium-server-standalone-3.141.5...
MacBook-Air-gh:~ ghghost$ java -version
java version "13.0.1" 2019-10-15
Java(TM) SE Runtime Environment (build 13.0.1+9)
Java HotSpot(TM) 64-Bit Server VM (build 13.0.1+9, mixed mode, sharing)
MacBook-Air-gh:~ ghghost$ java -jar /Users/ghghost/Downloads/selenium-server-standalone-3.141.59.jar
12:58:14.769 INFO [GridLauncherV3.parse] - Selenium server version: 3.141.59, revision: e82be7d358
12:58:14.991 INFO [GridLauncherV3.lambda$buildLaunchers$3] - Launching a standalone Selenium Server on port 4444
2020-11-03 12:58:15.184:INFO::main: Logging initialized @1194ms to org.seleniumhq.jetty9.util.log.StdErrLog
12:58:16.006 INFO [WebDriverServlet.<init>] - Initialising WebDriverServlet
12:58:16.285 INFO [SeleniumServer.boot] - Selenium Server is up and running on port 4444
```

Selenium



Это проект, в рамках которого разрабатывается серия программных продуктов с открытым исходным кодом (open source):

- Selenium WebDriver

- Selenium Server

- Selenium Grid

- Selenium IDE

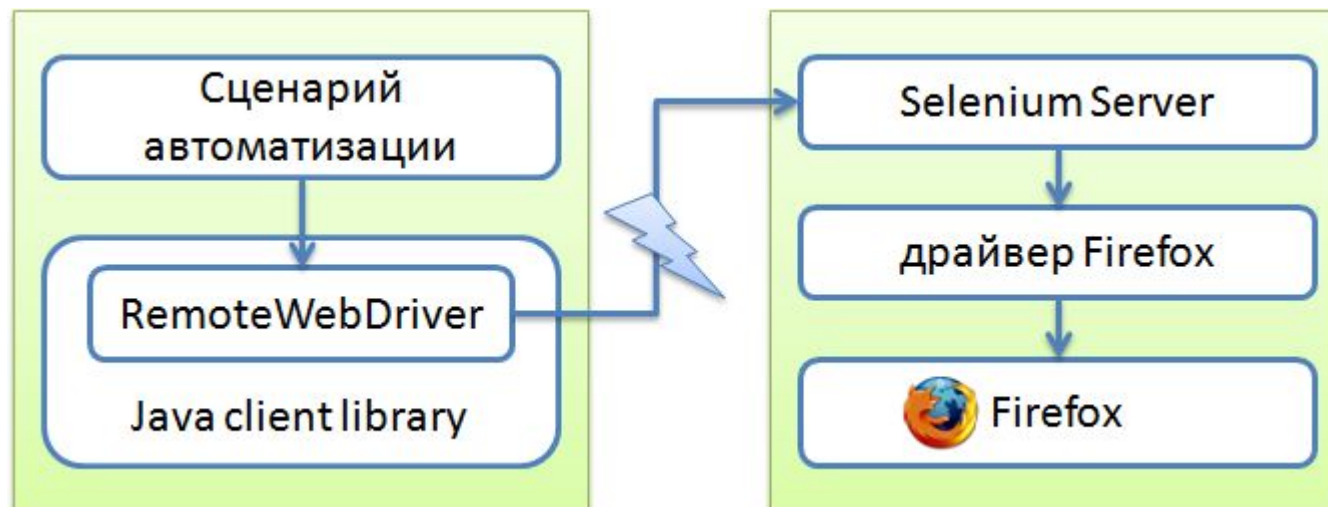
Webdriver

Драйвер браузера, то есть программная библиотека, которая позволяет разрабатывать программы, управляющие поведением браузера.

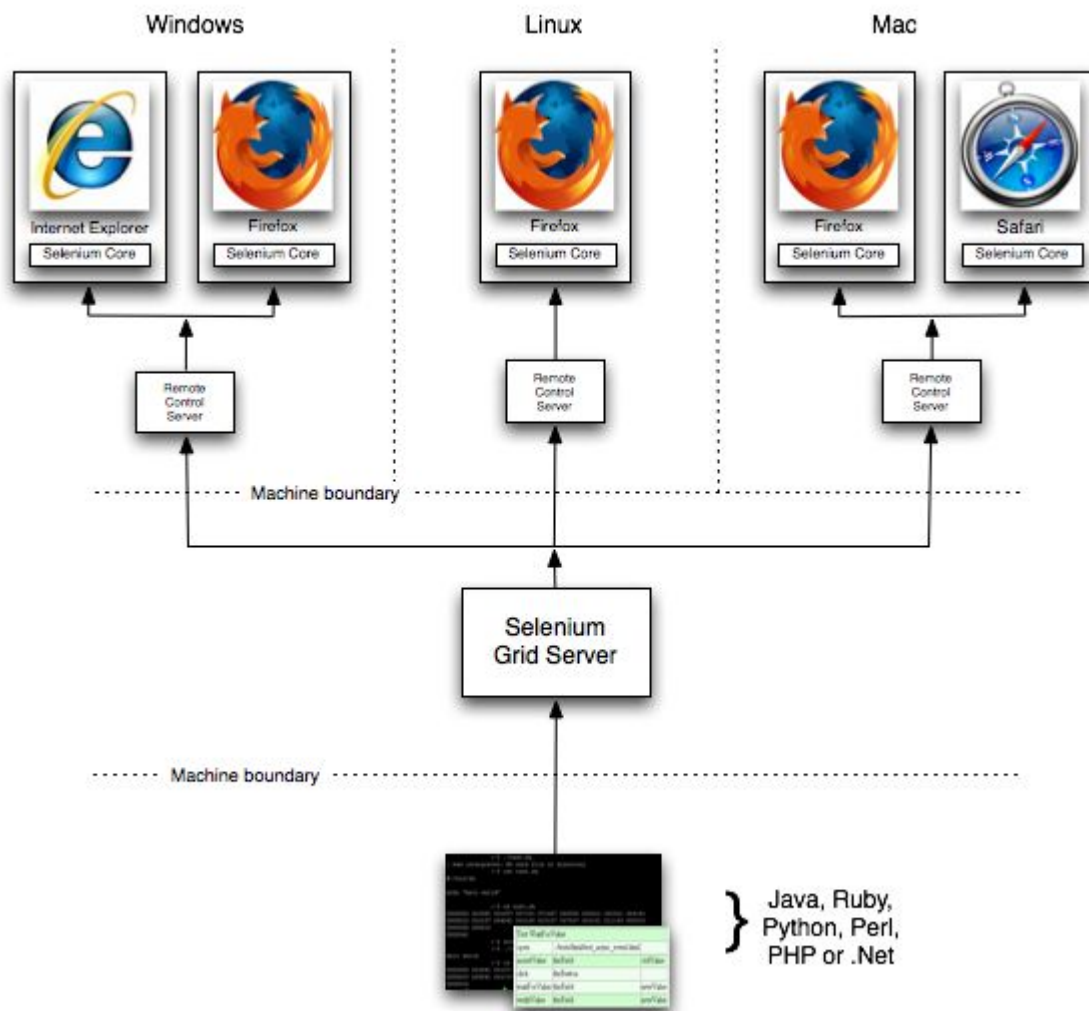
Selenium Server



Selenium Server – это сервер, который позволяет управлять браузером с удалённой машины, по сети.



Selenium Grid

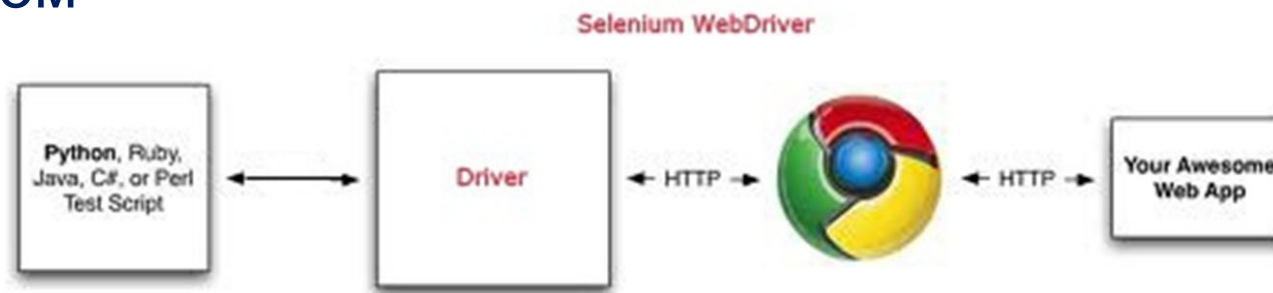


Selenium Grid – это кластер, состоящий из нескольких Selenium-серверов. Он предназначен для организации распределённой сети, позволяющей параллельно запускать много браузеров на большом количестве машин.

} Java, Ruby, Python, Perl, PHP or .Net

Взаимодействие с браузером и UI

Взаимодействие с браузером



Взаимодействие с UI

1. Найти элемент UI;
2. Произвести действие с элементом.



Поддерживаемые платформы

Операционные системы

Mac OS X, Linux, Windows



Браузеры

Firefox (up to 30.x)

IE 7-11

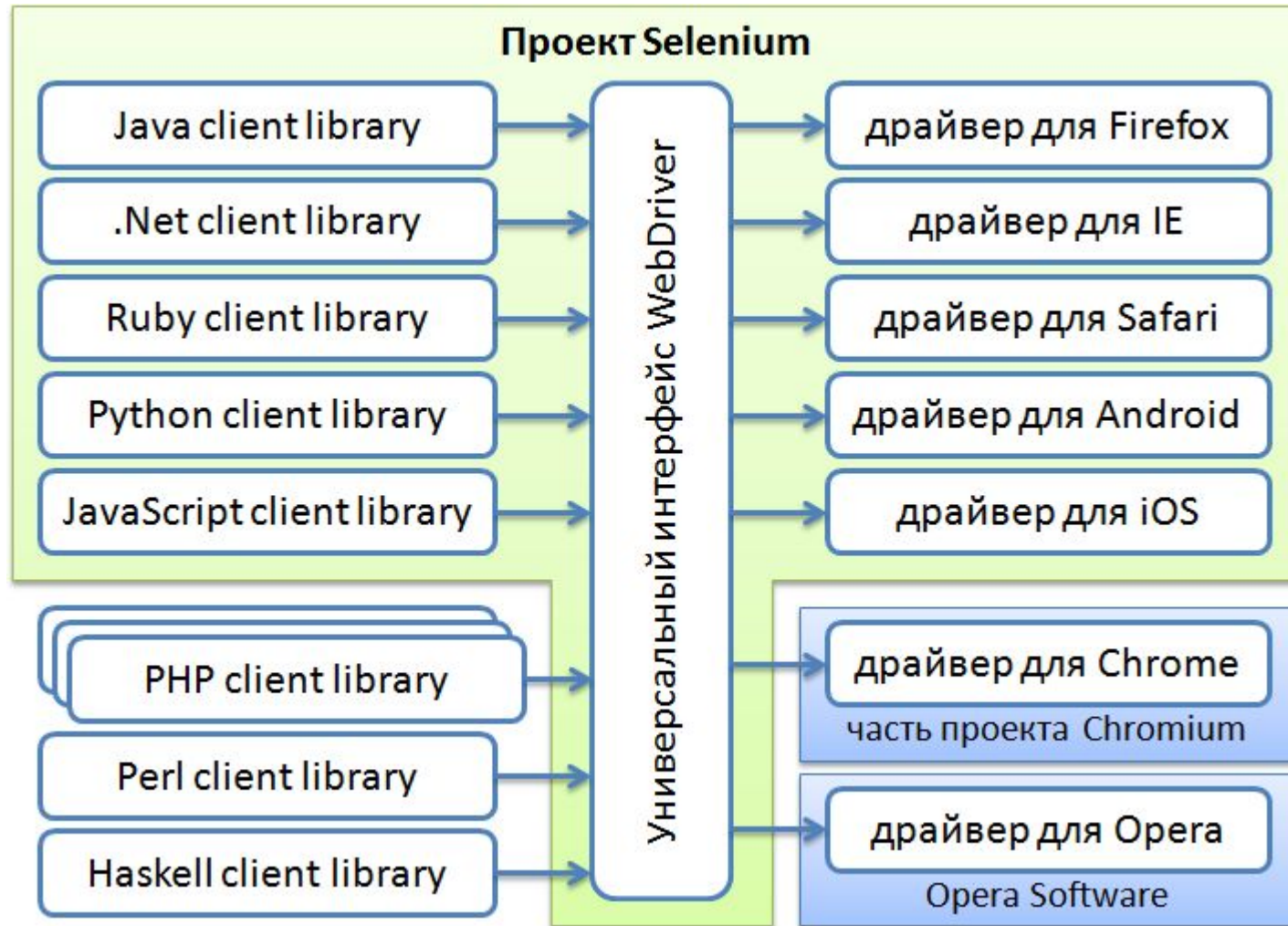
Safari (not stable)

Opera 8.x +

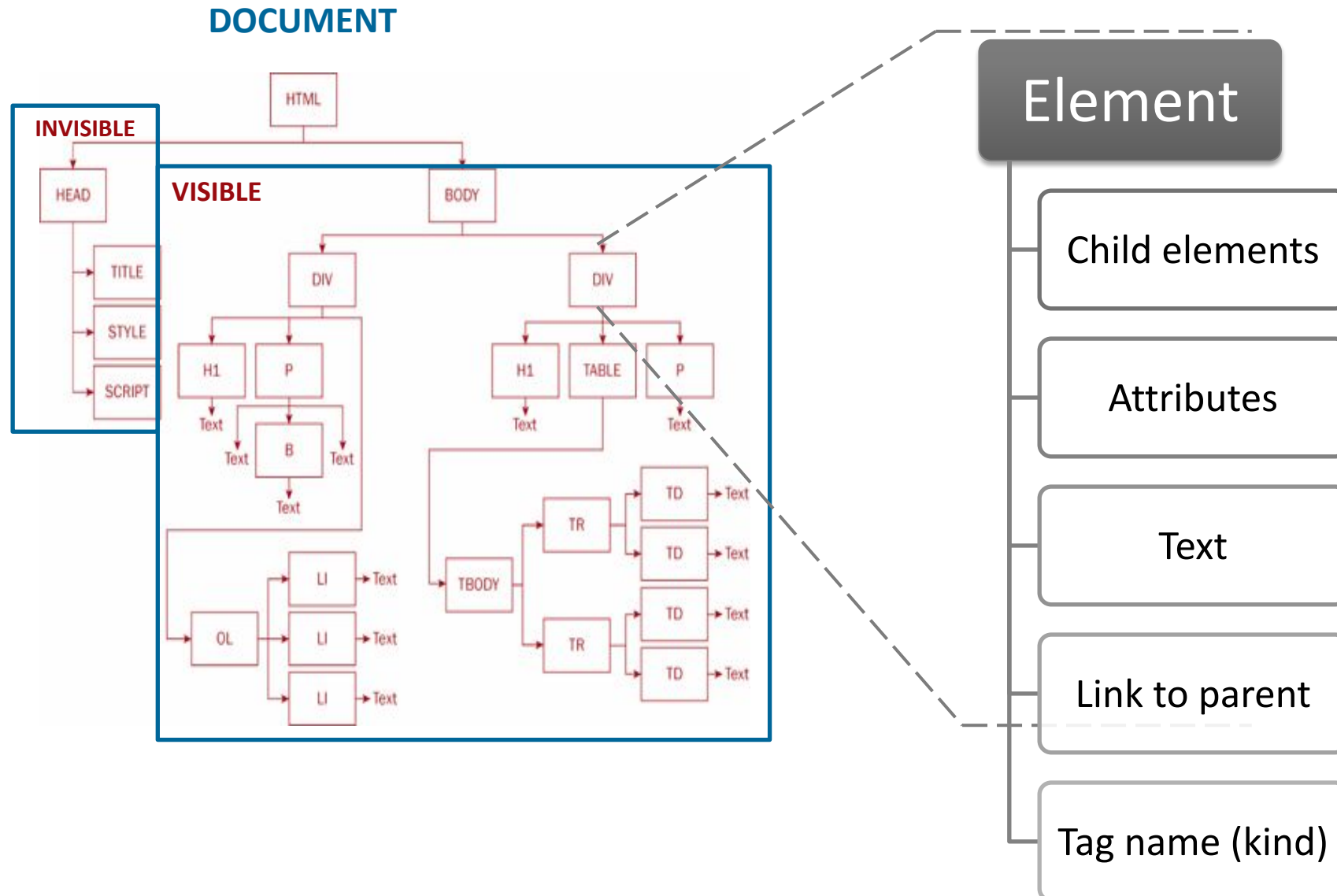
Google Chrome (all)



Компоненты (драйвера)



Document Object Model



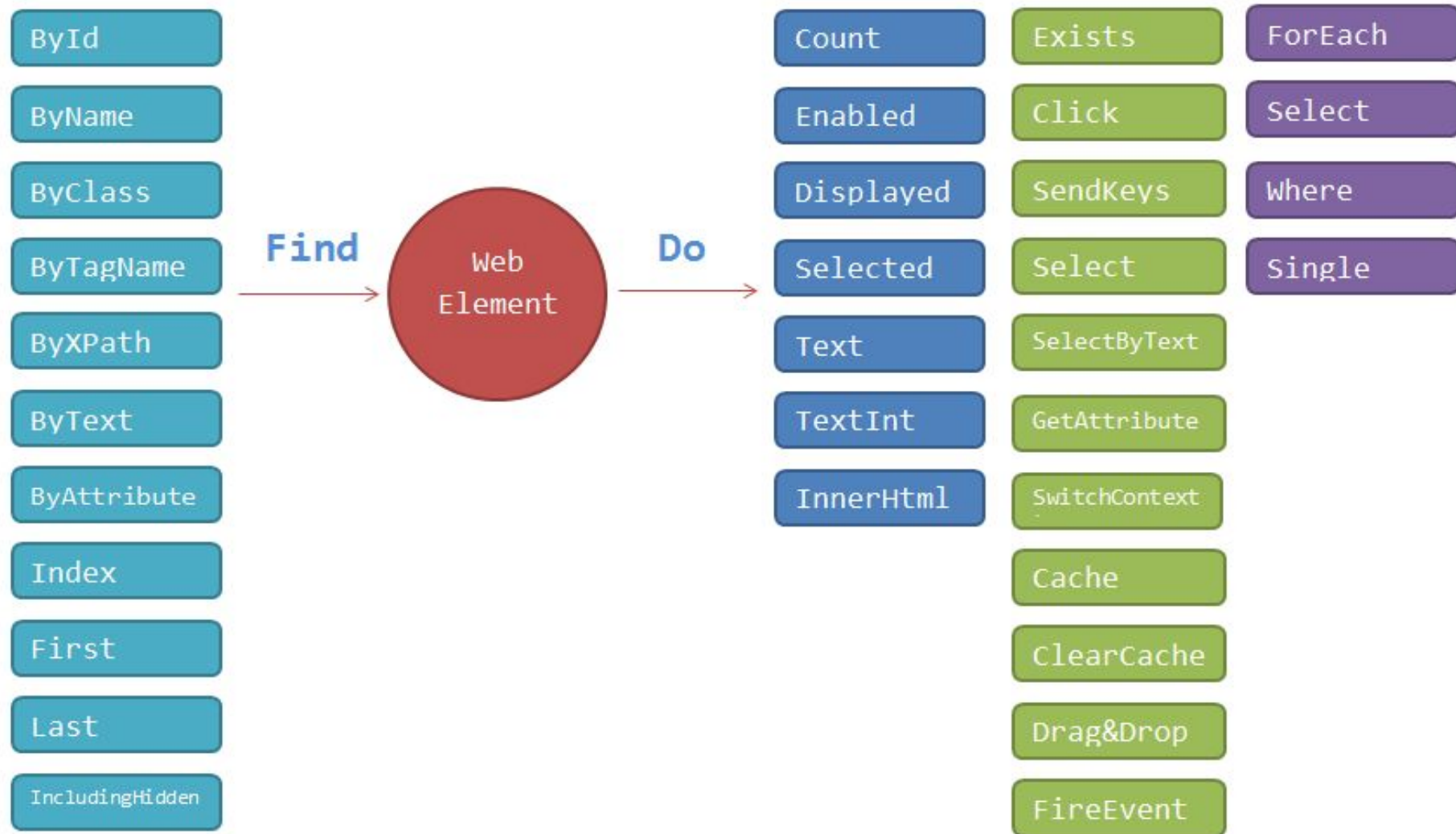
HTML DOM via JavaScript

```
1 <html>
2 <head>
3   ...
4 </head>
5 <body class="owaLgnBdy">
6 <form action="/owa/auth.owa" method="POST" name="loginForm">
7 <div id="pnlMain">
8   <div id="pnlHdr" class="header" >
9     
11  <div id="pnlLogin" class="mid">
12    <div id="pnlSec">
13      <span>Безопасность (отобразить пояснения)</span>
14      <input id="rdoPblc" type="radio" name="trusted" value="0"
15      <label for="rdoPblc">Это общедоступный или совместно испо
16      <input id="rdoPrvt" type="radio" name="trusted" value="1"
17      <label for="rdoPrvt">Это частный компьютер</label>
18    </div>
19    <div id="pnlLghtVrs">
20      <input id="chkBsc" type="checkbox" checked="true"></td>
21      <label for="chkBsc">Использовать облегченную версию Outlc
22      <span>Облегченная версия Outlook Web App включает меньшее
23      <a href="http://go.microsoft.com/fwlink/?LinkID=129362" i
24    </div>
25    <div id="pnlAuth" >
26      <label for="username">Адрес электронной почты:</label>
27      <input id="username" name="username" type="text" class="t
28      <label for="password">Пароль:</label>
29      <input id="password" name="password" type="password" />
30      <input type="submit" class="btn" value="Войти">
31    </div>
32  </div>
33  <div id="pnlFtr" class="footer">
34    <span>Подключено к Microsoft Exchange</span>
35    <span class="tdCopy">&copy; Корпорация Майкрософт (Microsoft
36  </div>
37 </div>
38 </form>
39 </body>
40 </html>
```

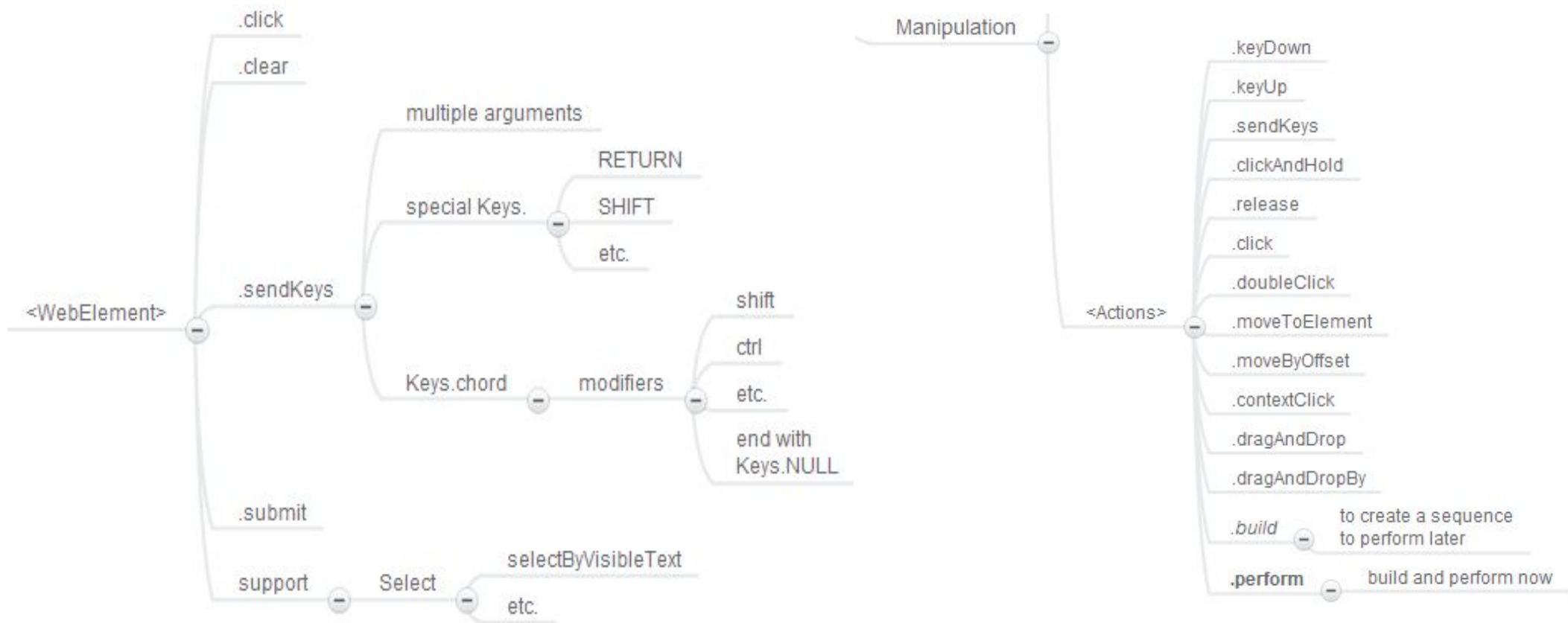
Functions can be combined in sequences:

`document.getElementById("pnlLogin")`
`.getElementsByTagName("input")[0]`

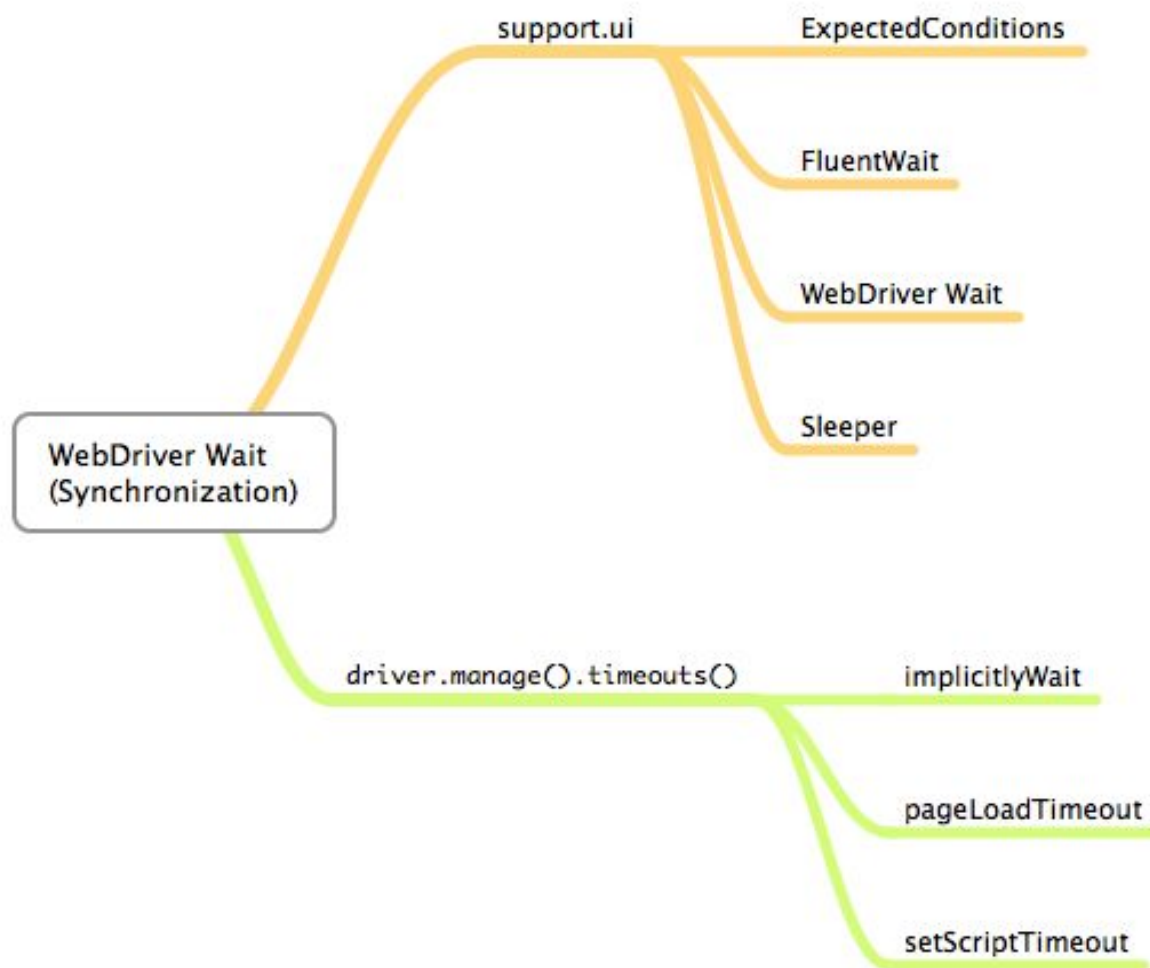
Как найти WebElement из Webdriver



Манипуляции с элементами



Синхронизация с UI



Selenium WebDriver

WebDriver - управление браузером. Выполнение тестов строится при помощи экземпляра этой сущности.

WebElement - сущность, представляющая собой абстракцию над веб-элементом (кнопки, ссылки, поля ввода и др.). WebElement инкапсулирует методы для взаимодействия пользователя с элементами и получения их текущего статуса.

By - Класс, содержащий статические методы для идентификации элементов. Абстракция над локатором веб-элемента. Этот класс инкапсулирует информацию о селекторе(например, CSS), а также сам локатор элемента, то есть всю информацию, необходимую для нахождения нужного элемента на странице.

Selenium WebDriver

WebElement element = driver.findElement(By.<критерий поиска>) -

Возвращает первый найденный элемент, удовлетворяющий, условию поиска

driver.findElements(By.<критерий поиска>) - Возвращает все элементы,
удовлетворяющие условию поиска

Основные методы поиска элементов:

- **driver.findElement(By.xpath(<XPath>))** – поиск элемента по Xpath
- **driver.findElement(By.id(<ID>))** – поиск элемента по ID
- **driver.findElement(By.name(<имя>))** – поиск элемента по значению атрибута name
- **driver.findElement(By.linkText(<текст ссылки>))** – поиск элемента по тексту ссылки

Selenium WebDriver

driver.navigate().to(<URL>) - переход по URL

driver.getTitle() - возвращает Title текущей страницы;

element.isDisplayed() - является ли элемент видимым

element.isEnabled() - является ли элемент доступным

element.isSelected() - является ли элемент выбранным (чекбокс, радиобаттон)

element.sendKeys(<последовательность символов>) - послать элементу последовательность символов. Текстовых или функциональных клавиш

Selenium. Ожидания

Explicit Wait

Explicit Waits (явное ожидание) – готовое решение с использованием класса `WebDriverWait`. Представляет собой фактически циклическое **ожидание указанного события**. По умолчанию “`WebDriverWait`” вызывает “`ExpectedCondition`” каждые 500 миллисекунд до тех пор, пока условие не будет удовлетворено.

Ожидаемые события:

- **`ExpectedConditions.visibilityOf(<элемент>)`** - видимость элемента
- **`ExpectedConditions.presenceOfElementLocated(<локатор>)`** - наличие элемента
- **`ExpectedConditions.textToBePresentInElement (<локатор>, <текст>)`** - текст элемента

Пример:

```
WebDriverWait wait = new WebDriverWait(driver, 10);
```

```
wait.until(ExpectedConditions.visibilityOf(element));
```

// Данный код будет либо выдаст исключение “`TimeoutException`” по прошествии 10 секунд, либо вернет найденный в течении 10 секунд элемент.

Explicit Wait

```
WebDriver driver = new FirefoxDriver();
driver.get("http://somedomain/url_that_delays_loading");
WebElement myDynamicElement = (new WebDriverWait(driver, 10))
    .until(new ExpectedCondition<WebElement>(){
        @Override
        public WebElement apply(WebDriver d) {
            return d.findElement(By.id("myDynamicElement"));
        }
    });
```

На практике рекомендуется использование `WebDriverWait` в сочетании с методами класса `ExpectedConditions`, которые позволяют сократить ожидание. Если элемент появился раньше времени, заданного при инициализации `WebDriverWait` - Selenium не будет ждать, а продолжит выполнение теста.

Implicit Wait

Неявное ожидание (Implicit Wait) устанавливается один раз в коде вне операции поиска и действительно до изменения. Значение по умолчанию - 0, т.е. никакого ожидания. Implicit Wait применяется ко *всем последующим* операциям поиска неявно (т.е. скрытно, без указания напрямую в методе поиска)

```
WebDriver driver = new FirefoxDriver();  
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);  
driver.get("http://somedomain/url_that_delays_loading");  
WebElement myDynamicElement =  
driver.findElement(By.id("myDynamicElement"));
```


Fluent Wait

Беглое ожидание (Fluent Wait) применяется к элементам, которые иногда появляются всего за 1 секунду, а иногда это занимает несколько минут. В этом случае лучше определить явное ожидание с помощью `FluentWait`, так как он будет пытаться искать такие элементы снова и снова, пока не найдет их или пока не истечет таймер. *`WebDriverWait` является типом `FluentWait`*, так как `WebDriverWait` расширяет `FluentWait` и имеет все возможности класса `FluentWait`, такие как возможность регулировать интервал опроса DOM, игнорировать исключения.

```
Wait<WebDriver> wait = new FluentWait<WebDriver>(driver)    .withTimeout(30,
SECONDS)
    .pollingEvery(5, SECONDS)
    .ignoring(NoSuchElementException.class);
WebElement foo = wait.until(new Function<WebDriver, WebElement>() {
public WebElement apply(WebDriver driver) {
    return driver.findElement(By.id("foo"));
}});
```

Ожидания. Пример кода на Selenium

```
public class Selenium2Example {  
    public static void main(String[] args) {  
        WebDriver driver = new FirefoxDriver();  
        driver.get("http://www.google.com");  
        // driver.navigate().to("http://www.google.com");  
        WebElement element = driver.findElement(By.name("q"));  
        element.sendKeys("Cheese!");  
        element.submit();  
        System.out.println("Page title is: " + driver.getTitle());  
        (new WebDriverWait(driver, 10)).until(new ExpectedCondition<Boolean>() {  
            public Boolean apply(WebDriver d) {  
                return d.getTitle().toLowerCase().startsWith("cheese!");  
            }  
        });  
        System.out.println("Page title is: " + driver.getTitle());  
        driver.quit();  
    }  
}
```

Паттерн Page object

Паттерн Page Object

- ▶ Решение проблемы избыточного дублирования кода при написании тестов
- ▶ Возможность совместной работы по тестам
- ▶ Упрощение поддержки кода при изменениях

Пример кода тестов на Selenium

```
@Test
public void loginTest(){
    WebDriver driver = new ChromeDriver();
    driver.get("http://www.google.com");
    driver.findElement(By.id("login")).sendKeys(...charSequences:"admin");
    driver.findElement(By.id("pass")).sendKeys(...charSequences:"password");
    assertEquals(actual:"Home Page", driver.getTitle());
}
```

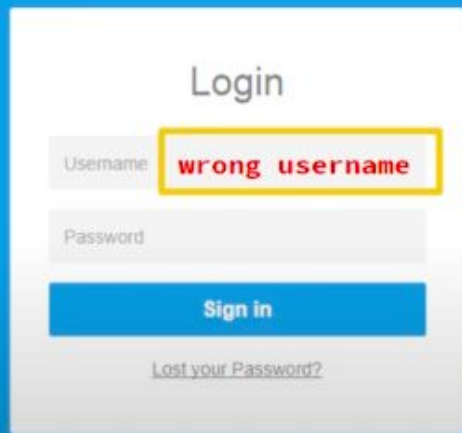
Пример кода тестов на Selenium - негативные сценарии

@Test

```
public void successLoginTest() {  
    driver.get("http://www.google.com");  
    driver.findElement(By.id("login")).sendKeys(...charSequences:"admin");  
    driver.findElement(By.id("pass")).sendKeys(...charSequences:"password");  
    assertEquals(actual:"Home Page", driver.getTitle());  
}
```

@Test

```
public void wrongLoginTest() {  
    driver.get("http://www.google.com");  
    driver.findElement(By.id("login")).sendKeys(...charSequences:"111");  
    driver.findElement(By.id("pass")).sendKeys(...charSequences:"password");  
    assertEquals(actual:"Home Page", driver.getTitle());  
}
```



Don't have an account? [Sign up here!](#)

Пример кода тестов на Selenium - МНОЖЕСТВО СХОЖИХ ТЕСТОВ

```
@Test
public void successLoginTest() {
    driver.get("http://www.google.com");
    driver.findElement(By.id("loginbtn")).sendKeys(...charSequences:"admin");
    driver.findElement(By.id("pass")).sendKeys(...charSequences:"password");
    assertEquals(actual:"Home Page", driver.getTitle());
}

@Test
public void wrongLoginTest() {
    driver.get("http://www.google.com");
    driver.findElement(By.id("loginbtn")).sendKeys(...charSequences:"111");
    driver.findElement(By.id("pass")).sendKeys(...charSequences:"password");
    assertEquals(actual:"Home Page", driver.getTitle());
}

@Test
public void wrongPasswordTest() {
    driver.get("http://www.google.com");
    driver.findElement(By.id("loginbtn")).sendKeys(...charSequences:"admin");
    driver.findElement(By.id("pass")).sendKeys(...charSequences:"111");
    assertEquals(actual:"Home Page", driver.getTitle());
}
```

Пример кода тестов на Selenium - применение паттерна Page Object

```
27  @Test
28  ▶ public void successfulLoginTest() {
29      loginPage.open();
30      assertTrue(loginPage.atPage());
31      loginPage.enterUserName("admin");
32      loginPage.enterPassword("changeme");
33      loginPage.clickLogin();
34      assertTrue(homePage.atPage());
35  }
36
37  @Test
38  ▶ public void wrongPasswordLoginTest() {
39      driver.get("https://lingualeo.com");
40      driver.findElement(By.id("login")).sendKeys(...charSequences: "admin");
41      driver.findElement(By.id("pass")).sendKeys(...charSequences: "wrongPass");
42      assertEquals(actual: "Something went wrong...", driver.getTitle());
43  }
44
```

LoginPage.java

3) Call methods

1) add methods

HomePage.java

2) Cut & Paste

Пример кода тестов на Selenium - применение паттерна Page Object

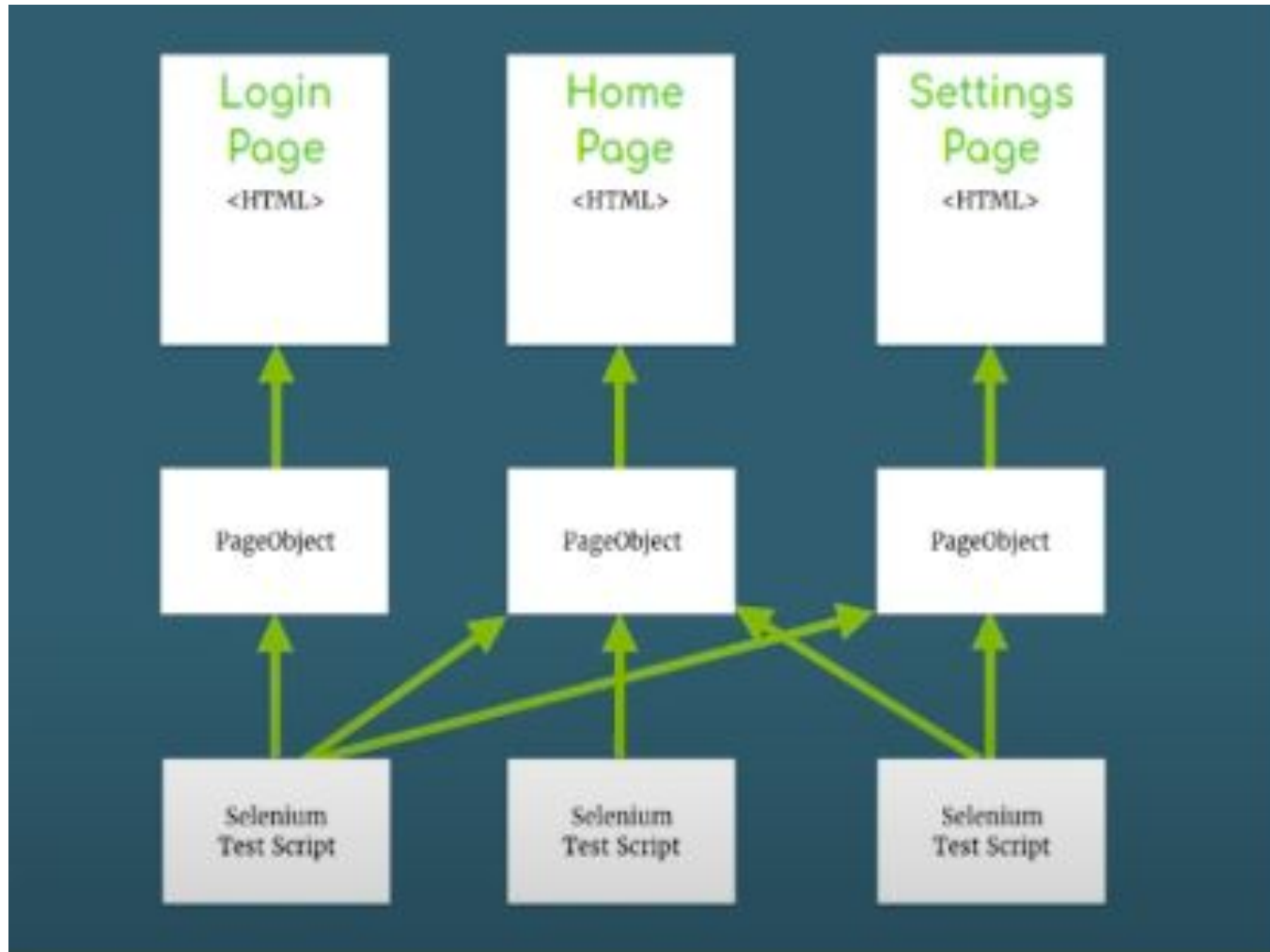
- 1) Создаем Java классы с именами страниц:
LoginPage.java , HomePage.java

Добавляем в них java методы, название которых
будет совпадать с действием на странице
- 2) Переместить код, который взаимодействует с
веб-элементами на странице (найти элемент,
ввести текст и т.д.) внутрь методов в Page Object
- 3) Создать объект Page Object

`loginPage = new loginPage ();`

через переменную loginPage вызывать методы,
которые мы поместили внутрь Page Object

Пример кода тестов на Selenium - применение паттерна Page Object



Пример кода тестов на Selenium - применение паттерна Page Object

```
import org.openqa.selenium.By
import org.openqa.selenium.WebDriver

public class LoginPage() {

    private WebDriver driver;
    LoginPage (WebDriver driver) {
        this.driver=driver;
    }

    void enter Username(String name) {
        driver.findElement(By.id("login")).sendKeys(...charSequences: "admin");
    }
    void enterPassword(String pass) {
        driver.findElement(By.id("pass")).sendKeys(...charSequences: "111");
    }
    void clickLogin() { }

    void open () { driver.get(https://google.com); }

    boolean atPage () {
        if (driver.getTitle().equals("Google")) {
            return true;
        } else {
            return false;
        }
    }
}
```

Пример кода тестов на Selenium - применение паттерна Page Object

```
36  
37  
38 ▶ @Test  
39 public void wrongPasswordLoginTest() {  
40     loginPage.open();  
41     assertTrue(loginPage.atPage());  
42     loginPage.enterUserName("admin");  
43     loginPage.enterPassword("wrongPassword");  
44     loginPage.clickLogin();  
45     assertTrue(homePage.atPage());  
46 }
```



Page Factory

Selenium: работаем с элементами страницы, используя @FindBy и PageFactory

Page Factory — это встроенная концепция объектной модели страницы для Selenium WebDriver. Концепция разделения хранилища объектов страниц и методов тестирования.

С помощью класса PageFactory мы используем аннотации **@FindBy** для поиска WebElement. Мы используем метод `initElements` для инициализации веб-элементов

@FindBy может принимать в качестве **атрибутов** **tagName** , **partialLinkText**, **name**, **linkText**, **id**, **css**, **className**, **xpath** .

Selenium: работаем с элементами страницы, используя @FindBy и PageFactory

```
public class AnnotatedSearchPage implements
SearchPage {
    @FindBy(id = "text")
    private WebElement searchField;

    @FindBy(css = "input[type=\"submit\"]")
    @CacheLookup
    private WebElement searchButton;

    @Override
    public void search(final String query) {
        searchField.sendKeys(query);
        searchButton.click();
    }

    @Override
    public void init(final WebDriver driver) {
        PageFactory.initElements(driver, this);
    }
}
```

Selenium: работаем с элементами страницы, используя @FindBy и PageFactory

- 1) В методе `init()` мы вызываем `PageFactory.initElements(driver, this);`. Драйвер не начинает искать элементы на странице сразу, а ищет их как только мы обращаемся к полю класса.
- 2) Строка `searchButton.click();` «превращается» в `driver.findElement(By.cssSelector("input[type=\"submit\"]")).click();`
- 3) Не нужно писать `driver.findElements(...)` по всему классу